

Memoria de Práctica Final

Análisis y Diseño de Algoritmos
Ramificación y Poda - Camino de coste mínimo

Nombre: Pau Sempere Martínez
DNI: 20520990E

Índice

1. Estructuras de datos	1
1.1. Nodo	1
1.2. Lista de nodos vivos	1
2. Mecanismos de poda	1
2.1. Poda de nodos no factibles	1
2.2. Poda de nodos no prometedores	2
3. Cotas pesimistas y optimistas	2
3.1. Cota pesimista inicial (inicialización)	2
3.2. Cota pesimista del resto de nodos	2
3.3. Cota optimista	3
4. Otros medios empleados para acelerar la búsqueda	3
5. Estudio comparativo de distintas estrategias de búsqueda	3
6. Tiempos de ejecución	4

1. Estructuras de datos

1.1. Nodo

El nodo representa un estado dentro de la búsqueda del camino en el laberinto. Se ha implementado mediante una estructura (struct) que almacena la posición actual (fila, columna), el coste real acumulado o longitud del camino (g), la estimación heurística o cota optimista (h), y el coste total estimado ($f = g + h$). También almacena el historial de movimientos mediante un string llamado camino. Además, se sobrecarga el operador > para que la extracción priorice el nodo más prometedor.

```
1 struct Nodo {
2     int fila, columna;
3     int g;           // Coste real acumulado (longitud del camino)
4     int h;           // Cota optimista (heurística)
5     int f;           // Coste total estimado (g + h)
6     string camino; // Secuencia de movimientos realizados
7
8     bool operator>(const Nodo &otro) const {
9         if (f == otro.f)
10             return g < otro.g;
11         return f > otro.f;
12     }
13 };
```

1.2. Lista de nodos vivos

Para la lista de nodos vivos (LNV) se ha utilizado una cola de prioridad (priority_queue) implementada como un min-heap. Siguiendo las estrategias de búsqueda de ramificación y poda, se ha optado por una **Estrategia dirigida o LC (Least Cost / Primero el mejor)**. El criterio principal es extraer el nodo con el menor valor de f (coste total estimado), garantizando una exploración inteligente del espacio de soluciones.

```
1 priority_queue<Nodo, vector<Nodo>, greater<Nodo>> cola_prioridad;
```

2. Mecanismos de poda

2.1. Poda de nodos no factibles

Se descartan los nodos que no cumplen las restricciones del problema (posiciones fuera de los límites de la matriz o casillas marcadas con 0). Adicionalmente, se aplica una poda por dominancia para evitar ciclos o recálculos: se descartan los nodos si el coste g para llegar a una casilla es mayor o igual al coste con el que ya se había visitado previamente esa misma casilla.

```
1 // Poda por factibilidad (fuera de limites o es una pared/
   inaccessible)
2 if (nueva_fila < 0 || nueva_fila >= n || nueva_col < 0 ||
    nueva_col >= m || laberinto[nueva_fila][nueva_col] == 0) {
```

```

3     nodos_no_factibles++;
4     continue;
5 }
6
7 int siguiente_g = actual.g + 1;
8 // Poda por dominancia
9 if (siguiente_g >= min_coste_real[nueva_fila][nueva_col]) {
10     nodos_no_factibles++;
11     continue;
12 }

```

2.2. Poda de nodos no prometedores

Tal y como establece la teoría, se puede eliminar un nodo si su cota optimista es peor que la mejor cota pesimista. En este caso, un nodo se considera no prometedor y se poda si su coste total estimado ($f = g + h$) es mayor o igual que el coste de la mejor solución encontrada hasta el momento (`mejor_coste`).

```

1 // Poda por cota (nodo no prometedor, excede o iguala el mejor
  // coste encontrado)
2 if (siguiente_f >= mejor_coste) {
3     nodos_no_prometedores++;
4     continue;
5 }

```

3. Cotas pesimistas y optimistas

3.1. Cota pesimista inicial (inicialización)

Idealmente, la cota pesimista inicial se obtiene mediante una solución voraz (greedy). Al no disponer de un sub-óptimo inicial rápido para este laberinto específico, la cota pesimista se inicializa a un valor representativo del infinito (`1e9`), indicando que cualquier solución factible encontrada será mejor.

```

1 int mejor_coste = 1e9; // Cota pesimista global inicializada a "
  infinito"

```

3.2. Cota pesimista del resto de nodos

La cota pesimista global (`mejor_coste`) no se actualiza expandiendo caminos sub-óptimos desde nodos intermedios, ya que lanzar algoritmos voraces desde cada celda penalizaría el tiempo de ejecución. En su lugar, la cota pesimista se actualiza de forma dinámica cada vez que la estrategia de búsqueda *Best-First* logra alcanzar la casilla de salida (un nodo hoja completo). Si el coste real (g) del camino hallado es estrictamente menor que la cota pesimista actual, esta se actualiza inmediatamente, permitiendo que la cola de prioridad pade de forma mucho más agresiva el resto de ramas vivas.

```

1 // Es nodo hoja (hemos llegado a la salida)?
2 if (actual.fila == n - 1 && actual.columna == m - 1)

```

```

3 {
4     nodos_hoja++;
5     if (actual.g < mejor_coste)
6     {
7         mejor_coste = actual.g; // Actualizacion de la cota
            pesimista global
8         mejor_camino_str = actual.camino;
9         sol_actualizada_hoja++;
10    }
11    continue;
12 }

```

3.3. Cota optimista

La cota optimista estima, a mejor, el valor que podría alcanzarse al expandir el nodo relajando las restricciones. Dado que el problema permite 8 movimientos (incluyendo diagonales), la métrica perfecta para la heurística es la Distancia de Chebyshev. Toma el máximo entre la diferencia absoluta de las coordenadas X y las coordenadas Y hasta el destino.

```

1 // Cota optimista
2 int calcular_heuristica(int fila, int columna, int dest_fila, int
    dest_columna) {
3     return max(abs(fila - dest_fila), abs(columna - dest_columna)
        );
4 }

```

4. Otros medios empleados para acelerar la búsqueda

Se ha implementado una matriz de memoria algorítmica (`min_coste_real`) que guarda el coste mínimo real (g) para llegar a cada celda del laberinto. Esto actúa como un mecanismo avanzado de poda por dominancia que evita la expansión redundante de ramas hacia zonas ya resueltas de manera óptima.

```

1 // Matriz de dominancia: guarda el menor coste 'g' encontrado
2 vector<vector<int>> min_coste_real(n, vector<int>(m, 1e9));
3
4 // Si es factible y prometedor, actualizamos dominancia
5 min_coste_real[nueva_fila][nueva_col] = siguiente_g;

```

5. Estudio comparativo de distintas estrategias de búsqueda

Para realizar esta comparativa y analizar el impacto real de la heurística en la estrategia de ramificación y poda, se ha evaluado una segunda configuración. Esta alternativa consiste en anular la cota optimista (haciendo que la función devuelva siempre 0). Al

hacer esto, el coste estimado f es igual al coste real g , lo que transforma la búsqueda LC original (A^*) en un algoritmo de Búsqueda de Coste Uniforme.

```

1 // Modificacion temporal para realizar el estudio comparativo
2 int calcular_heuristica(int fila, int columna, int dest_fila, int
  dest_columna) {
3     return 0; // Se desactiva la heuristica para evaluar Coste
      Uniforme
4 }

```

A continuación, se muestra el cuadro comparativo de ambas configuraciones de exploración:

Estrategia	Nodos Vivos (Generados)	Tiempo Respuesta (ms)
Estrategia LC con Cota Optimista	2294	0.41
Estrategia LC sin Cota Optimista (Coste Uniforme)	5948	0.87

Cuadro 1: Comparativa de distintas configuraciones de exploración para el fichero 100-bb.maze.

6. Tiempos de ejecución

Fichero de test	Tiempo (ms)
100-bb.maze	0.41
200-bb.maze	0.89
300-bb.maze	8.47
500-bb.maze	20.87
700-bb.maze	24.93
900-bb.maze	42.45
k01-bb.maze	30.03
k02-bb.maze	1593.58
k03-bb.maze	185.48
k05-bb.maze	669.95
k10-bb.maze	4235.78

Cuadro 2: Tiempos de ejecución requeridos (límite 15.000 ms).